

Second Brain ด้วย Vector Search

จากสมการถึงระบบจริง — ChromaDB · LanceDB · Cloudflare Edge

ARRA Oracle · Workshop 26 กรกฎาคม 2026

สารบัญ

บทนำ	3
คำตอบ 3 คำถามตั้งต้น (research สรุปร)	4
โครงหนังสือ (4 ภาค · 12 บท · demo ทุกบท)	4
1.1 ปัญหาที่ทุกคนมี	4
1.2 ติดตั้ง (1 คำสั่ง)	5
1.3 โค้ดเต็ม — 20 บรรทัด	5
1.4 เกิดอะไรขึ้นข้างใน (ภาพรวม 1 ย่อหน้า)	5
1.5 ลองรันจริง	5
1.6 ⚠ แต่เดี๋ยวก่อน — ผลภาษาไทยบางอันแปลกๆ	6
บทที่ 2 — ทำไมค้นภาษาไทยเพี้ยน (และแก้ยังไงใน 30 บรรทัด)	6
2.1 หลักฐานจากบทที่แล้ว	6
2.2 ใครกันแน่ที่ “เข้าใจภาษา”	7
2.3 แก่: เปลี่ยนคนแปะป้าย (เสีย bge-m3)	7
2.4 ผลหลังแก้ (query ชุดเดิมเป๊ะ)	8
2.5 ทำไม bge-m3 เข้าใจไทย (ย่อ — ฉบับเต็มดู deep-technical Ch19/22)	8
2.6 บทเรียนที่พกกลับบ้าน	8
บทที่ 3 — ค้นแบบมีเงื่อนไข: semantic + metadata พร้อมกัน	8
3.1 คำถามจริงไม่ได้มีแต่ความหมาย	8
3.2 metadata — ป้ายกำกับที่ติดตอน upsert	9
3.3 where — เงื่อนไขตอน query	9
3.4 ผลจริง (จาก demo3 — รันพิสูจน์แล้ว)	9
3.5 โบนัส: filter ด้วยเนื้อหา (where_document)	9
3.6 ⚠ กับดักที่ควรรู้ก่อนใช้จริง	10
3.7 เชื่อมกับของจริง	10
บทที่ 4 — Cosine Similarity: สมการเดียวที่ต้องรู้	10
4.1 ทวนภาพใหญ่ก่อน	10
4.2 คำนวณมือบนกระดาษ (2 มิติ)	10
4.3 โค้ด 5 บรรทัด (ทั้งฟังก์ชัน)	11
4.4 สเกลขั้นของจริง: 1024 มิติ ฟังก์ชันเดิมเป๊ะ	11
4.5 แล้ว vector database ทำอะไรกันแน่?	11
4.6 เกร็ดที่โปรรู้ (ย่อจาก deep-technical)	12
บทที่ 5 — Embedding มาจากไหน (และทำไมบางตัวบอดภาษาไทย)	12
5.1 การทดลอง: 3 โมเดล โจทย์ไทยชุดเดียวกัน	12
5.2 ทำไมถึงเป็นแบบนั้น — เข้าใจที่มาของ embedding	12

5.3 GAP สำคัญกว่าคะแนนดิบ	13
5.4 มิติไม่ใช่ตัวตัดสิน	13
5.5 เชื่อมกลับ ARRA (การเลือกที่มีหลักฐาน)	13
บทที่ 6 — ANN: ค้นหาน้ำในมิลลิวินาที (และเมื่อไหร่ไม่ต้องใช้)	14
6.1 คำถาม: “หา cosine สูงสุด” ในเน็ตล้านชิ้น ทำยังไงให้เร็ว	14
6.2 วัตถุจริง: brute force โต้งเง (numpy, 1024 มิติ)	14
6.3 HNSW — กราฟทางลัด (ANN)	14
6.4 ★ ผลวัตถุจริง — และบทเรียนข้อสัต์ยสองข้อ	14
6.5 กฎ scale-appropriate (ชิมของทั้งเล่ม)	15
6.6 เชื่อมกับ ARRA	15
บทที่ 7 — Hybrid Search: vector อย่างเดียวไม่พอ	15
7.1 จุดบอด 2 จุดของ vector (ที่เจอแน่ในเน็ตจริง)	15
7.2 BM25 — keyword scoring ใน 20 บรรทัด	15
7.3 ★ RRF — รวมสองโลกด้วย “อันดับ” ไม่ใช่คะแนน	16
7.4 ผลรันจริง (จาก notebook)	16
7.5 k=60 มาจากไหน	16
7.6 เชื่อม ARRA	16
บทที่ 8 — Ingest ทั้ง Vault: จากโพลเดอร์เน็ต → Second Brain	17
8.1 จากเน็ตทีละชิ้น → โพลเดอร์ทั้งใบ	17
8.2 Chunking ตามโครงสร้าง markdown	17
8.3 ★ Content-hash id — หัวใจของ idempotency	17
8.4 ผลรันจริง (พิสูจน์ครบสามโจทย์)	17
8.5 provenance — ตอบพร้อมที่มา	18
8.6 เชื่อม ARRA	18
บทที่ 9 — RAG: ให้ AI ตอบจากเน็ตของเรา (พร้อมอ้างอิง)	18
9.1 RAG ใน 1 บรรทัด	18
9.2 กติกา 2 ข้อที่มือใหม่พลาด	18
9.3 ⚠ กับดักจริงที่เจอตอนเขียน notebook (บันทึกไว้เพราะทุกคนจะเจอ)	19
9.4 ผลรันจริง	19
9.5 prompt ที่ใช้ (สั้นแต่ครบ)	19
9.6 เชื่อม ARRA + Claude	19
บทที่ 10 — เรื่องเล่าจริง: ทำไม ARRA ย้าย ChromaDB → LanceDB	20
10.1 ประวัติจริงจาก TIMELINE.md ของ ARRA	20
10.2 ผลรันเทียบจริง (corpus 200 chunks เดียวกัน, bge-m3 เดียวกัน)	20
10.3 ตัวตัดสินจริง: runtime ของแอป	20
10.4 บทเรียนใหญ่ที่สุดของบท	20
บทที่ 11 — วัดผลอย่างเป็นระบบ: Golden Set + Recall@k	20
11.1 ปัญหา: “รู้สึกที่ดีขึ้น” ไม่ใช่การวัด	21
11.2 Golden Set = ข้อสอบของระบบค้นหา	21
11.3 สอง metric ที่พอสำหรับเริ่มต้น	21

11.4 ★ ผลัดจริง — ตัวเลขที่จบทุกข้อสงสัย	21
11.5 ใช้ต่อในชีวิตจริง	21
บทที่ 12 (บทส่งท้าย) — Privacy & Local-First	21
12.1 สิ่งที่เราทำมาตลอดโดยไม่ได้พูดถึง	21
12.2 ownership ที่จับต้องได้	22
12.3 ต้นทุน (คำนวณใน notebook)	22
12.4 จบเล่ม — สิ่งที่คุณอ่านทำได้แล้วจริง (พิสูจน์ด้วย self-check ทั้ง 12 บท)	22
บทที่ 13 — ก้าวสู่ Production: LanceDB	22
13.1 เปิด second brain — embedded เหมือนเดิม	22
13.2 คั่น — ระบุ cosine ซัด (ตรงกับ ARRA lancedb.ts)	22
13.3 filter — SQL where แทน dict	22
13.4 ผลรัน (self-check ✅)	23
13.5 ต่างจาก Chroma ตรงไหน	23
บทที่ 14 — Hybrid ในตัว: LanceDB ทำ FTS + Vector เครื่องเดียว	23
14.1 สร้าง FTS index — คำสั่งเดียว	23
14.2 ★ Hybrid — vector + FTS + RRF ในคำสั่งเดียว	23
14.3 ผลรัน (self-check ✅)	23
14.4 บทเรียน	23
บทที่ 15 (บทส่งท้าย Production) — Time-Travel: ฟังก์ชันที่ Chroma ไม่มี	24
15.1 ทุกการแก้ = version ใหม่	24
15.2 ★ ย้อนดู + กู้คืน (undo จริง)	24
15.3 ผลรัน (self-check ✅)	24
15.4 ทำไม production เลือกสิ่งนี้	24
🎓 สรุปภาค Production	24
บทพิเศษ (Extras) — Vector Search ที่ Edge: Cloudflare Workers AI + Vectorize	24
X.1 Embedding ที่ edge — Workers AI	25
X.2 Vectorize — เก็บ+ค้นเวกเตอร์ที่ edge (v2 REST)	25
X.3 หัวใจ — ความรู้ portable	25

บทนำ

หนังสือ tool-anchored: ทุกบทมีโค้ดรันได้จริง (ChromaDB → bge-m3 → hybrid → ARRA production) แหล่งเนื้อหา: deep-technical/ 86 บท (~9,800 บรรทัด) + เดโมที่รันพิสูจน์แล้วใน book/demo/

คำตอบ 3 คำถามตั้งต้น (research สรุปร)

1. ควรอิงกับ tools ไหม? → ควร — ผู้เรียน (นักวิจัย/นักศึกษา ไม่ใช่ engineer) เรียนรู้จากของที่จับต้องได้ ทฤษฎีทุกบทต้องมีโค้ดรันตาม · หนังสือเดินคู่: แนวคิด (จาก deep-technical) + hands-on (ChromaDB)

2. ใช้ ChromaDB ดีไหม? → ดีสำหรับสอน ด้วยเหตุผลจริง: - pip install chromadb เดียวจบ — ไม่มี server, ไม่มี Docker (embedded เหมือน SQLite) - มี embedding ในตัว (เริ่มได้ทันที) + เสียบ embedder เองได้ (บทเรียน bge-m3) - **ARRA เองก็เริ่มจาก ChromaDB** (TIMELINE.md: Dec 2025 “FTS5 + ChromaDB hybrid = breakthrough”) → เรื่องเล่าสมบูรณ์: เรียนบน Chroma → เข้าใจว่าทำไม production ย้ายไป LanceDB - ⚠ จุดที่ต้องสอนคู่กัน: **default embedder อ่อนไทย** (พิสูจน์แล้ว demo1 คะแนนติดลบ/จับผิด → demo2 เสียบ bge-m3 แก้ได้) — นี่แหละคือบทเรียนที่มีค่าที่สุด

3. full setup + demo? → เสร็จแล้ว รันพิสูจน์แล้ว: - book/demo/setup.sh — uv venv + chromadb 1.5.9 ✓ - demo1_first_search.py — second brain 20 บรรทัด (โชว์ทั้งความสำเร็จและจุดอ่อนไทย) ✓ - demo2_thai_bge_m3.py — เสียบ bge-m3 ผ่าน Ollama → ไทยถูกทั้ง 3 query ✓

โครงหนังสือ (4 ภาค · 12 บท · demo ทุกบท)

ภาค 1 — เห็นมันทำงาน (demo-first, ไม่มีสมการ)

บท	เนื้อหา	demo	อิง deep-technical
1	Second brain แรก ใน 20 บรรทัด	demo1	Ch1 (แนวคิด)
2	ทำไมค้นไทยเพี้ยน → embedding model สำคัญกว่า DB	demo1 vs demo2	Ch19, Ch53
3	filter + metadata: ค้นแบบมีเงื่อนไข	demo3 (where filter)	Ch55, Ch61, Ch78

ภาค 2 — เข้าใจว่าทำไม (สมการเข้าเมื่อผู้อ่านเห็นผลแล้ว)

ewpage # บทที่ 1 — Second Brain แรกของคุณใน 20 บรรทัด

ภาค 1 · demo-first: บทนี้ไม่มีสมการเลย — เห็นมันทำงานก่อน แล้วค่อยไปเข้าใจว่าทำไม

1.1 ปัญหาที่ทุกคนมี

คุณจดโน้ตไว้เป็นร้อยเป็นพันไฟล์ — ใต้งานวิจัย สรุปรประชุม สูตรอาหาร รายการซื้อของ พอถึงเวลาต้องใช้ กลับหาไม่เจอ เพราะ คุณจำ “คำ” ที่เคยเขียนไม่ได้ จำได้แต่ “ความหมาย”

ค้นแบบเดิม (Ctrl+F / grep) เจอเฉพาะคำตรงเป๊ะ:

- คุณเขียนไว้ว่า “ประชุมกับอาจารย์ฝน เรื่อง workshop”
- คุณค้นว่า “นัดหมายกับใครบ้าง” → ไม่เจอ (ไม่มีคำว่า “นัดหมาย” ในโน้ต)

vector search แก่ตรงนี้: ค้นด้วย **ความหมาย** ไม่ใช่ตัวอักษร

1.2 ติดตั้ง (1 คำสั่ง)

```
pip install chromadb # หรือ: uv pip install chromadb
```

จบแค่นี้ — ไม่มี server ไม่มี Docker ไม่มี API key ChromaDB ทำงานแบบ *embedded* (ฝังในโปรแกรมเหมือน SQLite): ข้อมูลเป็นไฟล์เตอร์ในเครื่องคุณ

1.3 โค้ดเต็ม — 20 บรรทัด

```
import chromadb
```

```
# 1) เปิด "สมองที่สอง" - ข้อมูลเก็บที่ ./chroma_db ในเครื่องเรา
client = chromadb.PersistentClient(path="./chroma_db")
col = client.get_or_create_collection("second_brain")

# 2) ใส่โน้ต (upsert = รันซ้ำกี่ครั้งก็ไม่ซ้ำซ้อน)
col.upsert(
    ids=["n1", "n2", "n3"],
    documents=[
        "วิธีสอนนักศึกษาให้เข้าใจ vector search: เริ่มจาก cosine similarity ก่อน",
        "สูตรกาแฟ cold brew: กาแฟ 100g น้ำ 1L แช่ 18 ชั่วโมง",
        "ประชุมกับอาจารย์ฝน เรื่อง workshop วันที่ 26 กรกฎาคม",
    ],
    metadatas=[{"folder": "teaching"}, {"folder": "recipes"}, {"folder": "meetings"}],
)
```

```
# 3) ค้นด้วยความหมาย
res = col.query(query_texts=["นัดหมายกับใครบ้าง"], n_results=1)
print(res["documents"][0][0]) # → "ประชุมกับอาจารย์ฝน เรื่อง workshop..."
```

สามขั้นตอน: **เปิด** → **ใส่** → **ค้น** — ไม่มีคำว่า “นัดหมาย” ในโน้ตสักตัว แต่ระบบเข้าใจว่า “นัดหมาย” กับ “ประชุม” คือเรื่องเดียวกัน

1.4 เกิดอะไรขึ้นข้างใน (ภาพรวม 1 ย่อหน้า)

ตอน upsert ChromaDB แปลงข้อความแต่ละชิ้นเป็น **ตัวเลขหลายร้อยตัว** (เรียกว่า embedding) ที่เก็บ “ความหมาย” ไว้ — ข้อความความหมายใกล้เคียงกัน ตัวเลขก็ใกล้กัน ตอน query มันแปลงคำค้นเป็นตัวเลขแบบเดียวกัน แล้วหาโน้ตที่ตัวเลข “ใกล้ที่สุด” (วัด “ใกล้” ยังไง → บทที่ 4 มีสมการเดียวที่ต้องรู้รอก่อน)

1.5 ลองรันจริง

```
cd book/demo && ./setup.sh
.venv/bin/python demo1_first_search.py
```

ผลจริงจากเครื่องที่เขียนหนังสือเล่มนี้ (chromadb 1.5.9):

Q: การสอน (filter: folder=teaching เท่านั้น)
→ วิธีสอนนักศึกษาให้เข้าใจ vector search: เริ่มจาก cosine simi
→ Embedding คือการแปลงข้อความเป็นตัวเลขนัยมิติที่เก็บความหมาย

filter ทำงานสมบูรณ์ — ค้นเฉพาะโฟลเดอร์ teaching ได้ (จะลงลึกบทที่ 3)

1.6 ⚠ แต่เดี๋ยวก่อน — ผลภาษาไทยบางอันแปลกๆ

ถ้าคุณรัน demo1 เต็มๆ จะเห็นบางอย่างผิดปกติ:

Q: อยากสอนเรื่อง AI ค้นหา
0.531 ประชุมกับอาจารย์ผ่น เรื่อง workshop... ← ทำไมไม่ใช้โน้ตเรื่องสอน?!

Q: เครื่องดื่ม
-0.130 รายการซื้อของ... ← คะแนนติดลบ?

คำถามเรื่อง “การสอน” กลับได้โน้ตประชุมขึ้นก่อน และหลายคะแนน **ติดลบ** — ระบบไม่พัง แต่มีบางอย่างที่คุณต้องรู้เกี่ยวกับภาษาไทย

นี่แหละคือบทเรียนที่สำคัญที่สุดของหนังสือเล่มนี้ และคือเนื้อหาของบทถัดไป: ตัว vector database ไม่ใช่คนเข้าใจภาษา — **embedding model** ต่างหาก และ model ที่แถมมา เข้าใจภาษาอังกฤษเก่งกว่าไทยมาก (บทที่ 2: แก้อย่างไร — สบอยล์: bge-m3)

สรุปบทที่ 1

- second brain = คั่นโน้ตด้วย **ความหมาย** ไม่ใช่คำตรงเป๊ะ
- ChromaDB: pip install เดียว, embedded (เหมือน SQLite), ข้อมูลอยู่เครื่องเรา
- 3 ขั้น: PersistentClient → upsert → query
- ⚠ default embedder อ่อนภาษาไทย — เห็นด้วยตาแล้วใน demo → บทที่ 2 แก้

โค้ด: `book/demo/demo1_first_search.py` (รันพิสูจน์แล้ว) · ทยอยรู้เบื้องหลัง: *deep-technical Ch1*
ewpage

บทที่ 2 — ทำไมคั่นภาษาไทยเพี้ยน (และแก้อย่างไรใน 30 บรรทัด)

บทเรียนที่สำคัญที่สุดของเล่ม: **vector DB** ตัวไหนไม่สำคัญเท่า **embedding model** ตัวไหน

2.1 หลักฐานจากบทที่แล้ว

demo1 (ChromaDB + embedder ที่แถมมา) ตอบ query ไทย 3 ข้อ — ผิด 2 ใน 3:

Q: อยากสอนเรื่อง AI ค้นหา	
0.531 ประชุมกับอาจารย์ผ่น...	x (ควรได้โน้ตเรื่องการสอน)
Q: เครื่องดื่ม	
-0.130 รายการซื้อของ...	Δ (พอไปได้ แต่คะแนนติดลบ)
Q: นัดหมายกับใครบ้าง	
-0.130 รายการซื้อของ...	x (ควรได้โน้ตประชุม)

2.2 ใครกันแน่ที่ “เข้าใจภาษา”

แยกหน้าที่ให้ชัด — ระบบค้นหา 2 ส่วนที่คนมักเหมารวม:

ส่วน	หน้าที่	ตัวอย่าง
embedding model	แปลงข้อความ → ตัวเลข (ตัวที่ “เข้าใจภาษา”)	all-MiniLM, bge-m3
vector database	เก็บตัวเลข + หาตัวที่ใกล้เคียงที่สุด (เร็วๆ)	ChromaDB, LanceDB

ChromaDB ไม่ได้ผิดอะไรเลย — มันหา “ตัวเลขที่ใกล้เคียงที่สุด” ได้ถูกต้องสมบูรณ์ ปัญหาคือ **ตัวเลขที่ป้อนให้มันผิดตั้งแต่ต้น**: embedder ที่แถมมา (all-MiniLM-L6-v2) ถูกเทรนด้วยข้อความอังกฤษเป็นหลัก — เจอภาษาไทยแล้วแปลงความหมายได้ครึ่งๆ กลางๆ

เปรียบ: ห้องสมุดจัดชั้นหนังสือเก่งมาก แต่คนแปะป้ายหนังสืออ่านไทยไม่ออก — ต่อให้จัดชั้นเก่งแค่ไหน หนังสือก็ขึ้นผิดชั้น

2.3 แก่: เปลี่ยนคนแปะป้าย (เสีย bge-m3)

bge-m3 เป็น embedding model แบบ multilingual (100+ ภาษา รวมไทย) — เป็นตัวเดียวกับที่ ARR A Oracle ใช้ใน production จริง

รันในเครื่องเราผ่าน Ollama (ฟรี, local, privacy):

```
ollama pull bge-m3
```

แล้วบอก ChromaDB ให้ใช้ตัวนี้แทน — เขียน embedding function เองแค่ 10 บรรทัด:

```
import urllib.request, json
from chromadb.utils.embedding_functions import EmbeddingFunction

class OllamaBgeM3(EmbeddingFunction):
    def __call__(self, texts):
        req = urllib.request.Request(
            "http://localhost:11434/api/embed",
            data=json.dumps({"model": "bge-m3", "input": list(texts)}).encode(),
            headers={"Content-Type": "application/json"},
        )
        with urllib.request.urlopen(req, timeout=60) as r:
            return json.load(r)["embeddings"]

col = client.get_or_create_collection("second_brain_bge",
                                     embedding_function=OllamaBgeM3())
```

ที่เหลือเหมือนเดิมทุกอย่าง — upsert และ query ไม่ต้องแก้สักรหัส

2.4 ผลหลังแก้ (query ชุดเดิมเป๊ะ)

Q: อยากสอนเรื่อง AI ค้นหา

0.266 วิธีสอนนักศึกษาให้เข้าใจ vector search... ✓

Q: เครื่องดื่ม

0.043 รายการซื้อของ: นม ไข่ ขนมปัง กาแฟดริป ✓

-0.020 สูตรกาแฟ cold brew... ✓ (อันดับ 2 ก็เกี่ยว!)

Q: นัดหมายกับใครบ้าง

0.028 ประชุมกับอาจารย์ผ่น เรื่อง workshop... ✓

ถูกทั้ง 3 ข้อ — เปลี่ยนอย่างเดียวคือ embedding model, ไม่ได้แตะ database เลย

2.5 ทำไม bge-m3 เข้าใจไทย (ย่อ — ฉบับเต็มดู deep-technical Ch19/22)

- เทรนด้วยข้อความ 100+ ภาษา แบบ **contrastive**: จับคู่ประโยคความหมายเดียวกันข้ามภาษา ให้ตัวเลขออกมาใกล้กัน → “ประชุม” กับ “นัดหมาย” (และ “meeting”) เลยอยู่ย่านเดียวกัน
- มิติ 1024 (vs 384 ของตัวแถม) — พื้นที่เก็บความหมายละเอียดกว่า
- ผลข้างเคียงที่ได้ฟรี: **ค้นข้ามภาษาได้** — จดไทย ค้นอังกฤษ ก็เจอ

2.6 บทเรียนที่พกกกลับบ้าน

1. เลือก **embedding model** ก่อน เลือก database — model คือตัวเข้าใจภาษา
2. ภาษาไทย → ต้อง multilingual model (bge-m3 คือตัวที่พิสูจน์แล้วใน ARRA production)
3. ทดสอบด้วย **query ภาษาจริงของเราเอง** เสมอ — benchmark ภาษาอังกฤษบอกอะไรเราไม่ได้ (deep-technical Ch39: “คะแนน leaderboard อังกฤษดี ไม่การันตีไทยดี”)
4. อาการ “คะแนนติดลบ / จับคู่มั่ว” = สัญญาณ embedder ไม่เข้าใจภาษานั้น

สรุปบทที่ 2

- DB ไม่ได้เข้าใจภาษา — **embedding model เข้าใจ** (แยกสองส่วนนี้ให้ออก)
- default embedder อ่อนไทย → เสียบ bge-m3 ผ่าน Ollama = 10 บรรทัด แก้จบ
- พิสูจน์แล้วด้วย query ชุดเดียวกัน: ผิด 2/3 → ถูก 3/3

โค้ด: `book/demo/demo2_thai_bge_m3.py` (รันพิสูจน์แล้ว) · ลึกกว่า: deep-technical Ch2/7/19/22/53

ewpage

บทที่ 3 — ค้นแบบมีเงื่อนไข: semantic + metadata พร้อมกัน

“หาโน้ตเรื่องการสอน เฉพาะปีที่ไม่ใช่ฉบับร่าง” — ครั้งแรกคือความหมาย ครั้งหลังคือเงื่อนไข

3.1 คำถามจริงไม่ได้มีแต่ความหมาย

คำค้นในชีวิตจริงมักเป็นลูกผสม:

- “งานวิจัย vector search ทั้งหมด **หลังปี 2023**”
- “โน้ตประชุม ในโฟลเดอร์งาน ที่พูดถึง embedding”

- “แผนสอน **ที่ไม่ใช่ draft**”

ส่วนที่เป็น *ความหมาย* (“งานวิจัย vector search”) ให้ vector จัดการ ส่วนที่เป็น *เงื่อนไขอื่นๆ* (ปี > 2023, folder = งาน, draft = false) ให้ **metadata filter** จัดการ

3.2 metadata — ป้ายกำกับที่ติดตอน upsert

ตั้งแต่บทที่ 1 เราติด metadata ให้ทุกโน้ตอยู่แล้ว:

```
col.upsert(
  ids=["d1"],
  documents=["แผนสอน vector search สำหรับ workshop เดือนกรกฎาคม"],
  metadatas=[{"folder": "teaching", "year": 2026, "draft": False}],
)
```

metadata เป็นอะไรก็ได้ที่เป็น str / int / float / bool — โฟลเดอร์ ปี แท็ก สถานะ ผู้เขียน

3.3 where — เงื่อนไขตอน query

```
col.query(
  query_texts=["การสอนเรื่องค้นหาด้วยความหมาย"], # ← ความหมาย (vector)
  where={"$and": [                                # ← เงื่อนไข (metadata)
    {"folder": {"$eq": "teaching"}},
    {"year": {"$eq": 2026}},
    {"draft": {"$eq": False}},
  ]},
  n_results=3,
)
```

operator ที่ใช้ได้: \$eq \$ne \$gt \$gte \$lt \$lte \$in \$nin และประกอบด้วย \$and / \$or

3.4 ผลจริง (จาก demo3 — รันพิสูจน์แล้ว)

vault ทดลอง 6 โน้ต: แผนสอนจริง, แผนสอน draft, โน้ต SQL ปี 2025, งานวิจัย, โอเดียสอน, รายจ่าย

1) semantic ล้วน:

แผนสอน vector search สำหรับ workshop	← ✓
แผนสอน vector search ฉบับร่างแรก	← ปน draft มา
โน้ตสอน SQL พื้นฐานปีที่แล้ว	← ปนปีเก่ามา

2) + filter (teaching & 2026 & ¬draft):

แผนสอน vector search สำหรับ workshop	← ✓
บันทึกโอเดียสอน: ใช้เดโมก่อนค่อยลงสมการ	← ✓ (draft กับปีเก่าหายไปตามสั่ง)

semantic หาของ “เกี่ยว” — filter ตัดของ “ไม่เข้าเงื่อนไข” — สองอย่างทำงานพร้อมกันใน query เดียว

3.5 โบนัส: filter ด้วยเนื้อหา (where_document)

```
col.query(query_texts=[...], where_document={"$contains": "เดโม"})
```

บังคับว่าผลลัพธ์ต้อง *มีคำนี้จริงๆ* ในเนื้อหา — เป๊ะแบบ Ctrl+F ผสมกับความหมายแบบ vector (นี่คือรูปแบบอย่างง่ายของ “hybrid search” — เวอร์ชันเต็มรอบที่ 7)

คำ	เวกเตอร์
แมว	[0.9, 0.1]
ลูกแมว	[0.85, 0.15]
รถยนต์	[0.1, 0.95]

คิด $\cos(\text{แมว}, \text{ลูกแมว})$ ทีละขั้น:

```
dot    = 0.9×0.85 + 0.1×0.15      = 0.78
|แมว|  = √(0.9² + 0.1²)          = 0.906
|ลูกแมว| = √(0.85² + 0.15²)      = 0.863
cos     = 0.78 / (0.906 × 0.863)  = 0.998 ← เกือบ 1: ความหมายแทบเดียวกัน
```

ส่วน $\cos(\text{แมว}, \text{รถยนต์}) = 0.214$ — ทิศเกือบตั้งฉาก = คนละเรื่อง

4.3 โค้ด 5 บรรทัด (ทั้งฟังก์ชัน)

```
import math
```

```
def cosine(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    na = math.sqrt(sum(x * x for x in a))
    nb = math.sqrt(sum(x * x for x in b))
    return dot / (na * nb)
```

ไม่มีเวกเมนต์ ไม่มี AI ในฟังก์ชันนี้ — แค่คุณ บวก หาร

4.4 สเกลขึ้นของจริง: 1024 มิติ ฟังก์ชันเดิมเป๊ะ

demo4 ใช้ ฟังก์ชัน `cosine` ตัวเดียวกันข้างบน กับ embedding จริงจาก bge-m3 (ผลรันจริง):

มิติจริง: 1024

```
cos("แมวนอนบนโซฟา", "ลูกแมวหลับอยู่บนเก้าอี้") = 0.8181
cos("แมวนอนบนโซฟา", "ราคาน้ำมันดีเซลปรับขึ้น") = 0.3065
```

จุดที่ควรหยุดคิด: สองประโยคแมวไม่มีคำซ้ำกันเลยสักคำ (แมว/ลูกแมว, โซฟา/เก้าอี้, นอน/หลับ) แต่ $\cosine = 0.82$ — เพราะ bge-m3 แปลงให้ประโยคความหมายใกล้เคียงกันชั้ติสใกล้เคียงกัน (บทที่ 5: มันทำได้ยังไง)

4.5 แล้ว vector database ทำอะไรกันแน่?

ตอนนี้ตอบได้เต็มปาก:

ChromaDB (และ vector DB ทุกตัว) = ที่เก็บเวกเตอร์ + เครื่องหา cosine สูงสุดแบบเร็วมาก

`col.query(...)` ที่เข้ามา 3 บท ข้างในคือ: embed คำค้น → คำนวณ cosine กับโน้ตทุกตัว → เรียงคะแนน → คืน top-k (ที่ว่า “เร็วมาก” ทำยังไงเมื่อโน้ตเป็นล้าน — บทที่ 6: ANN)

หมายเหตุ: Chroma รายงานเป็น **distance** (ระยะ) = ประมาณ $1 - \text{similarity}$ — เลขน้อย = ใกล้ demo ของเราจึงพิมพ์ 1-distให้อ่านเป็นคะแนนความใกล้

4.6 เกร็ดที่โปรู้ (ย่อจาก deep-technical)

- ทำไมนิยม cosine ไม่ใช่ระยะทางตรงๆ (Euclidean)? — ตัดผลของ “ความยาวเอกสาร” ออก เหลือแต่ “ทิศ=ความหมาย” + เกลียวเชิงตัวเลขกว่า (Ch1, Ch50)
- embedder ยุคใหม่ปล่อยเวกเตอร์ normalize มาแล้ว ($\|v\|=1$) → cosine = dot product เฉยๆ → เร็วขึ้นอีก (Ch1 §1.5)

สรุปบทที่ 4

- ความใกล้เคียงของความหมาย = $\cos(A,B) = A \cdot B / (|A||B|)$ — สมการเดียว, โค้ด 5 บรรทัด
- คำนวณมือได้ใน 2 มิติ · สเกลขึ้น 1024 มิติ ฟังก์ชันเดิมเป๊ะ (พิสูจน์แล้ว: แมว 0.82 vs น้มนม 0.31)
- vector DB = ที่เก็บเวกเตอร์ + หา cosine สูงสุดเร็วๆ — แค่นั้นจริงๆ

โค้ด: `book/demo/demo4_cosine_by_hand.py` (รันพิสูจน์แล้ว) · ลีกรว่า: deep-technical Ch1/8/50
ewpage

บทที่ 5 — Embedding มาจากไหน (และทำไมบางตัวบอภาษาไทย)

บทที่ 4 บอกว่า “ประโยคความหมายใกล้ → เวกเตอร์ชี้ทิศใกล้” — บทนี้ตอบว่าใครสอนให้มันเป็นแบบนั้น พร้อมผลทดลองจริงที่ช็อกที่สุดในเล่ม

5.1 การทดลอง: 3 โมเดล โจทย์ไทยชุดเดียวกัน

เอา embedder 3 ตัวที่รันได้ฟรีใน Ollama มาเจอคู่ประโยค 5 คู่ — 3 คู่ “ควรใกล้” (พาราเฟรสไทย, ความหมายเกี่ยว, ไทย↔อังกฤษ) และ 2 คู่ “คนละเรื่อง” (ผลรันจริง):

โมเดล	คู่ใกล้ (เฉลี่ย)	คู่ไกล (เฉลี่ย)	GAP
bge-m3 (1024-dim)	0.718	0.325	0.393 ✓
qwen3-embedding:0.6b (1024-dim)	0.559	0.264	0.296
nomic-embed-text (768-dim)	0.805	1.000	-0.195 ✗

อ่านแถวสุดท้ายอีกครั้งซ้ำๆ: nomic ให้ “ประชุมกับอาจารย์เรื่องเวิร์กช็อป” กับ “สูตรต้มยำกุ้งน้ำข้น” cosine = **1.000** — มันมองสองประโยคไทยที่ไม่เกี่ยวกันเลยเป็น *เวกเตอร์เดียวกันเป๊ะ*

5.2 ทำไมถึงเป็นแบบนั้น — เข้าใจที่มาของ embedding

embedding model คือ neural network ที่ถูก **เทรน** ด้วยวิธีที่เรียกว่า **contrastive learning**:

ป้อนคู่ประโยค “ความหมายเดียวกัน” ล้านๆ คู่ → บังคับให้เวกเตอร์ออกมาใกล้กัน
ป้อนคู่ประโยค “คนละเรื่อง” (negatives) → บังคับให้เวกเตอร์ออกมาไกลกัน
ทำซ้ำหลายพันล้านครั้ง → โมเดล “จัดระเบียบพื้นที่ความหมาย” ได้เอง

โมเดลจะเก่งได้เฉพาะ **ภาษาที่อยู่ในข้อมูลเทรน**:

- **nomic-embed-text**: เทรนด้วยอังกฤษเป็นหลัก → เจอไทยแล้ว “อ่านไม่ออก” ทุกประโยคไทย เลยถูกบีบไปกองที่มุมเดียวกันของพื้นที่ → $\cos$ ต่อกัน ≈ 1.0 หมด (อาการนี้มีชื่อ: *anisotropy* — เวกเตอร์กระจุกในกรวยแคบ, deep-technical Ch43)
- **bge-m3**: เทรน contrastive ด้วย 100+ ภาษา **ข้ามภาษา** (“ประชุม” จับคู่ “meeting”) → พื้นที่ความหมายครอบภาษาไทยจริง → คู่ไกลถูกดันออกไป 0.325 → GAP กว้าง

5.3 GAP สำคัญกว่าคะแนนดิบ

สังเกตว่า nomic ให้คู่ไกลคะแนน สูงสุด (0.805) — ถ้าดูเลขเดียวจะหลงคิดว่ามันเก่ง! แต่ค้นหาจริงคือการ **จัดอันดับ**: ของเกี่ยวข้องชนะของไม่เกี่ยวข้อง

เกณฑ์ที่ถูก: **GAP = (คะแนนคู่เกี่ยวข้อง) – (คะแนนคู่ไม่เกี่ยวข้อง)** ยิ่งกว้างยิ่งดี GAP ติดลบ = ของไม่เกี่ยวข้องชนะของเกี่ยวข้อง = คั่นพัง 100% ไม่ว่าคะแนนดิบสวยแค่ไหน

นี่คือเหตุผลที่บทที่ 2 บอกให้ “ทดสอบด้วย query ภาษาเราเอง” — คะแนนดิบและ benchmark ภาษาอังกฤษหลอกเราได้ แต่ GAP บนข้อมูลจริงของเราไม่หลอก

5.4 มิติไม่ใช่ตัวตัดสิน

qwen3 กับ bge-m3 มิติเท่ากัน (1024) แต่ GAP ต่างกัน (0.296 vs 0.393) ส่วน nomic 768 มิติพังไม่ใช่เพราะมิติน้อย — เพราะ **ข้อมูลเทรนไม่มีไทย**

ลำดับความสำคัญตอนเลือก embedder: ภาษาที่ครอบ > คุณภาพข้อมูลเทรน > มิติ

5.5 เชื่อมกลับ ARRA (การเลือกที่มีหลักฐาน)

arra-oracle-v3 รองรับ 3 ตระกูลนี้จริง (KNOWN_DIMS: nomic 768, bge-m3 1024, qwen3 1024-4096) และเลือก **bge-m3 เป็น default** — ตอนนี้น่าเห็นแล้วว่าทำไม: มันคือตัวเดียวในสามตัวที่ GAP กว้างพอสำหรับ vault ไทย+อังกฤษปนกัน

โบนัสของ bge-m3 ที่ demo แถวที่ 3 พิสูจน์: คู่ **ไทย↔อังกฤษ** (“แมวนอนบนโซฟา” ↔ “The cat is sleeping on the couch”) ก็อยู่ฝั่ง “ใกล้” ได้ — จัดไทย คั่นอังกฤษ เจอ

สรุปบทที่ 5

- embedding เกิดจาก **contrastive learning**: ดันคู่ความหมายเดียวกันเข้า คู่คนละเรื่องออก
- โมเดลเก่งเฉพาะภาษาที่ถูกเทรน — nomic เจอไทย → ทุกประโยคกองรวม ($\cos \approx 1.0$, anisotropy)
- ตัดสินด้วย **GAP** ไม่ใช่คะแนนดิบ: bge-m3 0.393 ✓ · qwen3 0.296 · nomic **-0.195** ✗
- มิติไม่ใช่ตัวตัดสิน — ภาษาที่ครอบสำคัญกว่า · ARRA เลือก bge-m3 ด้วยเหตุผลนี้

โค้ด: `book/demo/demo5_embedder_shootout.py` (รันพิสูจน์แล้ว) · ลีangkwa: *deep-technical Ch2/7/19/22/37/43*

ewpage

บทที่ 6 — ANN: ค้นหาในมิลลิวินาที (และเมื่อไหร่ไม่ต้องใช้)

จบบท 2 — บทนี้วัดของจริงทุกตัวเลข (notebook ch06_ann_benchmark.ipynb รันซ้ำได้)

6.1 คำถาม: “หา cosine สูงสุด” ในโน้ตล้านชิ้น ทำยังไงให้เร็ว

วิธีตรงไปตรงมา (brute force): เทียบ query กับทุกโน้ต แล้วเรียงคะแนน — ได้คำตอบเป๊ะ 100% แต่เวลาโตตามจำนวนโน้ตแบบเส้นตรง — $O(N)$

6.2 วัดจริง: brute force โดยังไง (numpy, 1024 มิติ)

1,000 โน้ต → 0.03 ms
10,000 โน้ต → 0.65 ms
100,000 โน้ต → 8-10 ms
200,000 โน้ต → ~17 ms

เซอร์ไพรส์แรก: แสพโน้ตใช้เวลาแค่ ~10 ms — numpy (SIMD, deep-technical Ch44) เร็วกว่าที่คนส่วนใหญ่คิดมาก

6.3 HNSW — กราฟทางลัด (ANN)

เมื่อโน้ตเป็นล้านหรือ query ถี่มาก → ANN: ไม่เทียบทุกตัว แต่ “เดินกราฟ” จากจุดหนึ่ง กระโดดไปจุดที่ใกล้กว่าเรื่อยๆ — เหมือนถามทาง ไม่ใช่เคาะประตูทุกบ้าน

ChromaDB ใช้ HNSW ให้อัตโนมัติ พร้อม “ปุ่มหมุน” สามตัว:

```
col = client.create_collection('bench', metadata={
    'hsw:space': 'cosine',
    'hsw:search_ef': 200,          # ตอนค้น: สูง = แม่นขึ้น/ช้าลง
    'hsw:construction_ef': 200,   # ตอนสร้าง: สูง = กราฟคุณภาพดี
    'hsw:M': 32,                  # จำนวนเพื่อนต่อจุดในกราฟ
})
```

6.4 ★ ผลวัดจริง — และบทเรียนข้อสัต์ยสองข้อ

ทดลองบนเวกเตอร์สุ่ม 20,000 ตัว (โจทย์ที่ยากที่สุดของ ANN — ไม่มีโครงสร้างกลุ่มเลย):

	เวลา/query	top-5 ตรงกับคำตอบจริง
brute force (numpy)	2.9 ms	5/5 (นิยาม)
HNSW ef=50 (default-ish)	3.9 ms	2/5 ⚠
HNSW ef=200	12.6 ms	4-5/5 ✓

บทเรียน 1 — ANN คือ “approximate” จริงๆ: ef ต่ำ → เร็วแต่พลาดครั้งหนึ่ง! ปุ่มหมุน ef คือ trade-off recall ↔ ความเร็วของจริง ไม่ใช่ทฤษฎีในหนังสือ

บทเรียน 2 — scale ของเราอาจไม่ต้องใช้ ANN เลย: ที่ 20k โหนด brute force ยังเร็วกว่า HNSW ด้วยซ้ำ (2.9 vs 12.6 ms) เพราะ overhead ของกราฟ+เรียก DB — ANN คุ่มเมื่อล้านโหนดขึ้นไป หรือ latency ทุก ms มีค่า

6.5 กฎ scale-appropriate (ธิมของทั้งเล่ม)

โน้ตหมิ่น-แสน (personal vault): brute force / HNSW default → เร็วพอทั้งคู่ (ms)

โน้ตล้าน+ (องค์กร): HNSW จูน ef/M ตาม recall ที่ต้องการ

โน้ตพันล้าน (บริษัทใหญ่): sharding + quantization (deep-technical Ch25/48)

อย่า over-engineer: ระบบที่ซับซ้อนที่สุดไม่ใช่ระบบที่ดีที่สุด — ระบบที่พอดีกับ **scale** จริงต่างหาก

6.6 เชื่อมกับ ARRA

ARRA ใช้ LanceDB ที่มี IVF/HNSW ในตัว — แต่ประเด็นเดียวกัน: corpus ส่วนตัว ระดับหมิ่น-แสน ความเร็วไม่ใช่คอขวด (embed query ต่างหากที่ช้าสุด ~30-80ms, deep-technical Ch44) → พลังงานควรไปลงที่ **คุณภาพ embedding (บทที่ 2/5)** กับ **hybrid (บทที่ 7)** ไม่ใช่จูน ANN

สรุปบทที่ 6

- brute force = $O(N)$ แต่ numpy เร็วมาก: แส้นโหนด ~10ms → personal ไม่ต้อง ANN ก็ได้
- HNSW = กราฟทางลัด · ปุ่มหมุน ef/M = recall ↔ speed (วัดแล้ว: ef ต่ำพลาต 3/5!)
- ที่ 20k โหนด brute force ชนะ HNSW ด้วยซ้ำ — **scale-appropriate** เสมอ
- คอขวดจริงของ personal vault คือ embed ไม่ใช่ search

Notebook: [book/notebooks/ch06_ann_benchmark.ipynb](#) (execute ผ่าน, self-check ). ลีกว่า : deep-technical Ch3/17/44

ewpage

บทที่ 7 — Hybrid Search: vector อย่างเดียวไม่พอ

เปิดภาค 3 (ระบบจริง) — สร้าง hybrid engine ด้วยมือทั้งตัวใน notebook [ch07_hybrid_search.ipynb](#)

7.1 จุดบอด 2 จุดของ vector (ที่เจอแน่ในโน้ตจริง)

1. รหัส/ชื่อเฉพาะ: “PR #2740”, “ESP32”, “มาตรา 44” — embedding จับความหมายรวม แต่ตัวเลข/รหัสเป๊ะๆ อาจ dilute หายในเวกเตอร์ (บทที่ 5: pooling เฉลี่ยทุก token)
 2. คำว่า “ไม่”: $\cos(\text{“มีน้ำตาล”}, \text{“ไม่มีน้ำตาล”})$ สูงมาก — ต่างแค่ token เดียว topic กลบ
- ทั้งสองอย่างคือความถนัดของ **keyword search** (ตรงเป๊ะ, boolean NOT) ที่โลกใช้มา 40 ปี

7.2 BM25 — keyword scoring ใน 20 บรรทัด

สูตรที่ FTS5 (และ search engine แทบทุกตัว) ใช้:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 (1 - b + b \frac{|d|}{\text{avgdl}})}$$

- **IDF:** คำหายาก (เช่น “#2740”) ได้น้ำหนักมาก — คำที่มีทุก doc แทบไม่นับ
- **f(t,d)** แบบอิมตัว: คำซ้ำ 10 ครั้งไม่ได้เต็ม 10 เท่า (k_1 คมความอิม)
- **ปรับตามความยาว doc (b):** doc ยาวไม่ได้เปรียบ

notebook เขียนสูตรนี้จริงใน 20 บรรทัด — รันแล้ว “PR #2740” ขึ้นอันดับ 1 ทันที (IDF ของ “#2740” สูงมาก)

7.3 ★ RRF — รวมสองโลกด้วย “อันดับ” ไม่ใช่คะแนน

ปัญหาถ้ารวมคะแนนตรงๆ: cosine $\in [-1, 1]$ แต่ BM25 $\in [0, \infty)$ — คนละ scale บวกกันไม่ได้

RRF แก่ด้วยการใช้อันดับ (rank) ซึ่ง scale-free เสมอ:

$$\text{RRF}(d) = \sum_r \frac{1}{k + \text{rank}_r(d)}, \quad k = 60$$

```
def rrf(rank_lists, k=60):
    scores = {}
    for ranks in rank_lists:
        for pos, doc in enumerate(ranks, 1):
            scores[doc] = scores.get(doc, 0) + 1 / (k + pos)
    return sorted(scores.items(), key=lambda x: -x[1])
```

หลักฐานเชื่อมกับ production: fusedScore ใน arra-oracle-v3 = 0.016393 = 1/61 พอดี — คือ doc ที่ได้อันดับ 1 จาก ranker เดียว (1/(60+1)) · สูตรใน notebook กับใน production คือตัวเดียวกัน

7.4 ผลรันจริง (จาก notebook)

Q: “PR #2740” → BM25 จีบเป๊ะ · RRF top-1 = โน้ต PR #2740 ✓ (fused 0.0328 = 1/61+1/61 ชนะทั้งคู่)

Q: “บอร์ดสำหรับสอน IoT” → vector จีบความหมาย (ESP32/ไมโครคอนโทรลเลอร์) · RRF ✓

hybrid ไม่ใช่ “เพื่อเหนียว” — มันชนะเพราะสองระบบถนัดคนละเขต แล้ว RRF ปล่อยให้แต่ละตัวเด่นในเขตตัวเอง

7.5 k=60 มาจากไหน

งานวิจัย (Cormack 2009) พบว่า k=60 robust ข้าม dataset: - k เล็ก → อันดับต้นถ่วงแรงมาก (เชื่อ top-1 สุดๆ) - k ใหญ่ → เฉลี่ยแบนราบ - 60 = จุดสมดุลที่ใช้ได้แทบทุกงานโดยไม่ต้อง tune (ARRA ก็ใช้ค่านี้)

7.6 เชื่อม ARRA

ARRA hybrid = FTS5 (BM25 จริง, SQLite) + LanceDB (vector) + RRF k=60

mode ให้เลือก: hybrid (default) / fts / vector — ตรงกับที่เพิ่งสร้างเองทั้งตัว

ต่างแค่ scale: ARRA ใช้ inverted index จริง (เร็วระดับหมื่น doc) แทน loop Python — สูตรเดียวกันเป๊ะ

สรุปบทที่ 7

- vector พลาด: รหัสเบาะ + negation → BM25/keyword ถนัดพอดี → hybrid
- BM25 = IDF × ความถี่ในตัว × ปรับความยาว (เขียนเอง 20 บรรทัดใน notebook)
- ☆ RRF ใช้อันดับ (scale-free) k=60 — สูตรตรงกับ ARRA production ($1/61 = 0.016393$ พิสูจน์แล้ว)
- hybrid ชนะเพราะสองระบบถนัดคนละเขต

Notebook: `ch07_hybrid_search.ipynb` (execute  3 asserts) · ลีangkwa: *deep-technical Ch4/11/34/56/60*

ewpage

บทที่ 8 — Ingest ทั้ง Vault: จากโฟลเดอร์โน้ต → Second Brain

notebook `ch08_ingest_vault.ipynb` — pipeline จริง รันซ้ำได้ไม่พัง

8.1 จากโน้ตทีละชิ้น → โฟลเดอร์ทั้งใบ

ของจริงคือโฟลเดอร์ .md เป็นร้อยไฟล์ที่เปลี่ยนทุกวัน — pipeline ต้องตอบ 3 โจทย์: 1. แยกไฟล์เป็นชิ้นกันได้ (chunking) 2. รันซ้ำแล้วไม่ duplicate ไม่ embed ซ้ำ (idempotent — เพราะ sync ทุกวัน) 3. แก้ไฟล์นิดเดียว → จ่ายแค่ส่วนที่เปลี่ยน

8.2 Chunking ตามโครงสร้าง markdown

หัวข้อ (#, ##) คือรอยตัดธรรมชาติที่คนเขียนแบ่งความคิดไว้ให้แล้ว:

```
def chunk_markdown(text, source):  
    # ตัดที่ทุกบรรทัดที่ขึ้นต้นด้วย '#' → 1 section = 1 chunk  
    # เก็บ heading + source ติดไปเป็น metadata (provenance)
```

1 ไฟล์ → N chunk · แต่ละ chunk รู้ว่าตัวเองมาจากไฟล์ไหน หัวข้ออะไร → ค้นเจอแล้วอ้างอิงกลับได้

8.3 ☆ Content-hash id — หัวใจของ idempotency

```
def chunk_id(c):  
    return hashlib.sha256(f"{c['source']}|{c['heading']}|  
{c['text']}").encode().hexdigest()[:16]
```

id = hash ของเนื้อหา (หลักเดียวกับ git): - เนื้อหาเดิม → id เดิม → upsert ทับตัวเอง = ไม่ duplicate
- เช็ค id ก่อน embed → เนื้อหาเดิมไม่ต้อง embed ซ้ำ = ประหยัด (embed คือส่วนที่แพงสุด) - เนื้อหาเปลี่ยน → id ใหม่ → embed เฉพาะชิ้นนั้น

8.4 ผลรันจริง (พิสูจน์ครบสามโจทย์)

รอบ 1:	เพิ่ม 5 · ซ้ำม 0	(ingest ครั้งแรก)
รอบ 2 (รันซ้ำ):	เพิ่ม 0 · ซ้ำม 5	← idempotent!

หลังเพิ่ม section: เพิ่ม 1 · ซ้ำม 5 ← จ่ายเฉพาะส่วนใหม่
ค้น "วิธีชงกาแฟเข้มๆ" → เจอ section ใหม่ทันที ← freshness

8.5 provenance — ตอบพร้อมที่มา

ผลค้นทุกชิ้นแนบ source + heading:

Q: ต้องเตรียมอะไรไปสอน

📁 workshop-plan.md → อุปกรณ์
โน้ตบุ๊ก ติดตั้ง Python + Jupyter ล่วงหน้า...

นี่คือรากของ “คำตอบที่ verify ได้” — บทที่ 9 จะส่งต่อให้ LLM cite ตามนี้

8.6 เชื่อม ARRA

ARRA ทำ pipeline เดียวกันนี้ที่ scale จริง: batch embed ครั้งละ 50 + retry 3 ครั้ง + timeout 30s + fallback chain (Ollama → Gemini → CF) — โครงเหมือน notebook เป๊ะ แค่เพิ่มความทนทาน (deep-technical Ch51/52 มีทุกรายละเอียด)

สรุปบทที่ 8

- ingest = chunk (ตาม # heading) → content-hash id → idempotent upsert
- รันซ้ำ: เพิ่ม 0 · แก้ไฟล์: จ่ายแค่ chunk ใหม่ · ของใหม่ค้นเจอทันที — พิสูจน์แล้วทั้งสาม
- ทุก chunk มี provenance (ไฟล์+หัวข้อ) → พร้อมส่งให้ LLM cite (บทที่ 9)

Notebook: `ch08_ingest_vault.ipynb` (execute  · ลึกลงว่า: deep-technical Ch12/45/51/52
ewpage

บทที่ 9 — RAG: ให้ AI ตอบจากโน้ตของเรา (พร้อมอ้างอิง)

notebook `ch09_rag_cite.ipynb` — RAG ครบวงจร รวมถึงสิ่งที่สำคัญที่สุด: รู้จักบอก “ไม่พบ”

9.1 RAG ใน 1 บรรทัด

Retrieve โฉนดที่เกี่ยวข้อง → ประกอบ context พร้อมแหล่ง → **Generate**: LLM เรียบเรียงตอบ + cite

LLM ไม่ต้องจำความรู้ของเรา — โฉนดคือความจริง LLM คือผู้เรียบเรียง (deep-technical Ch42: memory > parameters)

9.2 กติกา 2 ข้อที่มีมือใหม่พลาด

ข้อ 1 — threshold: ANN คำนวณ top-k เสมอ แม้ไม่มีอะไรเกี่ยวข้อง (สไลด์แผนที่คำ: “ใกล้สุดในบรรดาที่มี”) → score ต่ำกว่าเกณฑ์ต้อง **ไม่ป้อน LLM** ไม่งั้น LLM ตอบมั่วจากขยะ

ข้อ 2 — แนบ source ทุกชิ้น: [workshop-plan.md] แผน workshop วันที่ 26... → LLM cite ตามได้
→ คนตรวจย้อนได้ → เชื่อถือได้

9.3 ⚠ กับดักจริงที่เจอตตอนเขียน notebook (บันทึกไว้เพราะทุกคนจะเจอ)

Chroma default วัดระยะแบบ L2 ไม่ใช่ cosine! — ตอนใช้ embedding function ของเราเอง คะแนน 1-distance ที่ได้คือ 1-L2² (เพี้ยนไปทั้งสเกล: 0.57 จริงแสดงเป็น 0.13) → ต้องระบุเองตอนสร้าง collection:

```
col = client.create_collection('rag_vault', embedding_function=BgeM3(),  
                               metadata={'hnsw:space': 'cosine'})
```

ARRA production ก็ระบุชุดแบบเดียวกัน: `.distanceType('cosine')` ใน `lancedb.ts` — **อย่าเชื่อ default เรื่อง distance metric** ตรวจสอบ

9.4 ผลรันจริง

Q: workshop วันไหน แล้วต้องเตรียมอะไรบ้าง

🤖 Workshop วันที่ 26 กรกฎาคม (workshop-plan.md) และต้องเตรียมอุปกรณ์คือ
โน้ตบุ๊กติดตั้ง Python และ Jupyter ส่วนหน้า พร้อม... (workshop-plan.md)

Q: ราคาหุ้นวันนี้เป็นยังไง

🤖 ไม่พบข้อมูลเรื่องนี้ใน vault ครับ
(retrieval คืบ 0 ขึ้นหลัง threshold — ไม่ป้อน LLM เลย)

- คำตอบแรก: ข้อเท็จจริงถูก + **cite ไฟล์จริง** — verify ได้
- คำตอบสอง: **abstain** — ระบบที่ยอมบอก “ไม่รู้” น่าเชื่อกว่าระบบที่ตอบทุกอย่าง

9.5 prompt ที่ใช้ (ส่วนตัวครับ)

ตอบคำถามจากบันทึกด้านล่างเท่านั้น ห้ามเดาข้อมูลนอกบันทึก
อ้างอิงชื่อไฟล์ในวงเล็บท้ายประโยคที่ใช้ข้อมูลนั้น เช่น (workshop-plan.md)
ถ้าบันทึกไม่มีคำตอบ ให้บอกว่าไม่พบข้อมูล

สามบรรทัด สามหน้าที่: ground (ห้ามเดา) · cite (อ้างไฟล์) · abstain (ยอมรับว่าไม่มี)

9.6 เชื่อม ARRA + Claude

ในระบบจริง Claude Code ทำหน้าที่ generate + ตัดสินใจ (จะค้นไหม ค้นกี่รอบ) เอง ส่วน ARRA คือ retrieve ที่แม่นยำ+เร็ว+แนบ provenance — แบ่งหน้าที่เดียวกับ notebook นี้เป๊ะ (deep-technical Ch75/80/81: context assembly, adaptive retrieval, abstention)

สรุปบทที่ 9

- RAG = retrieve (threshold + source) → generate (ground + cite + abstain)
- ⚠ Chroma default = L2 — ระบุ hnsw:space: cosine เสมอ (ARRA: distanceType('cosine'))
- พิสูจน์แล้ว: ตอบพร้อม cite ไฟล์จริง + บอก “ไม่พบ” เมื่อถามนอก vault
- prompt 3 บรรทัด: ห้ามเดา · อ้างไฟล์ · ยอมรับว่าไม่มี

Notebook: `ch09_rag_cite.ipynb` (execute ✅ 4 asserts) · ลีangkwa: deep-technical Ch42/75/80/81

ewpage

บทที่ 10 — เรื่องเล่าจริง: ทำไม ARRA ย้าย ChromaDB → LanceDB

เปิดภาค 4 (สู่ production) · notebook `ch10_chroma_to_lancedb.ipynb` รันทั้งสองระบบเทียบกันจริง

10.1 ประวัติจริงจาก TIMELINE.md ของ ARRA

- Dec 2025: ARRA เริ่มต้นด้วย “FTS5 + ChromaDB hybrid” — จุด breakthrough แรก
- ต่อมา: ย้าย vector store ไป LanceDB — ไม่ใช่เพราะ Chroma “แย่” แต่เพราะบริบทเปลี่ยน

10.2 ผลรันเทียบจริง (corpus 200 chunks เดียวกัน, bge-m3 เดียวกัน)

ingest: Chroma 44 ms · LanceDB 10 ms
query: Chroma 1.3 ms · LanceDB 2.2 ms
top-1: ตรงกันเป๊ะ ✓ (สมการเดียวกัน — บทที่ 4)

ที่ scale นี้ ต่างกันไม่มึน — ตัวตัดสินจริงไม่ใช่ความเร็ว

10.3 ตัวตัดสินจริง: runtime ของแอป

	ChromaDB	LanceDB
runtime	Python-first (ยุคนี้ต้องมี Python sidecar)	Rust core ฝังใน process — JS/TS เรียกตรง
storage	ภายใน	Lance columnar + versioned (time-travel)
เหมาะกับ	เรียน/prototype Python	ฝังในแอปที่ไม่ใช่ Python

ARRA เป็นแอป Bun/TypeScript → LanceDB ฝังใน runtime ตัวเองได้ ไม่ต้องมี Python sidecar → ย้ายเพราะ fit ไม่ใช่เพราะแพ้ benchmark · โปรเจกต์ Python → Chroma คือคำตอบที่ถูกแล้ว

10.4 บทเรียนใหญ่ที่สุดของบท

self-check บังคับว่า **top-1** ของสองระบบต้องตรงกัน — และมันตรงจริง เพราะทุกอย่างที่เรียนมา (embedding, cosine, hybrid, eval) ติดตัวคุณ ไม่ติดเครื่องมือ DB เปลี่ยนได้เสมอ ความรู้ไม่ต้องเปลี่ยนตาม

Notebook: `ch10_chroma_to_lancedb.ipynb` (execute  · ลีกว่า: deep-technical Ch45/64, TIMELINE.md

ewpage

บทที่ 11 — วัดผลอย่างเป็นระบบ: Golden Set + Recall@k

หัวใจของ “วัดผลได้อย่างมั่นใจ” · notebook `ch11_golden_set_eval.ipynb`

11.1 ปัญหา: “รู้สึกที่ดีขึ้น” ไม่ใช่การวัด

ทั้งเล่มเราอ้างว่า bge-m3 ดีกว่า — ผู้อ่านไม่ควรเชื่อจนกว่าจะวัดเองได้

11.2 Golden Set = ข้อสอบของระบบค้นหา

(query, เฉลย doc ที่เกี่ยวข้องจริง) ที่คนตัดสินไว้ — สร้างครั้งเดียว ใช้ได้ตลอด:

```
GOLDEN = [( 'อยากสอนเรื่อง AI ค้นหาข้อมูล', {0, 4} ),  
            ( 'นัดหมายกับใครไว้บ้าง', {1} ), ... ] # 7 ข้อ บน corpus 8 โน้ต
```

11.3 สอง metric ที่พอสำหรับเริ่มต้น

$$\text{Recall@ } k = \frac{|\text{เฉลยที่ติด top- } k|}{|\text{เฉลยทั้งหมด}|} \quad \text{MRR} = \frac{1}{N} \sum \frac{1}{\text{อันดับเฉลยตัวแรก}}$$

11.4 ★ ผลวัดจริง — ตัวเลขที่จบทุกข้อสงสัย

โมเดล	Recall@3	MRR	รายชื่อ
MiniLM (ตัวแถม)	0.36	0.37	××××✓✓× (ผิด 4/7)
bge-m3	0.93	1.00	✓✓✓✓✓✓✓ (ถูกทุกข้อ, MRR 1.0 = เฉลยขึ้นอันดับ 1 เสมอ)

คำว่า “ดีกว่า” กลายเป็นตัวเลข: **0.93 vs 0.36** บนข้อสอบไทยของเราเอง — ใครไม่เชื่อ รันซ้ำได้

11.5 ใช้ต่อในชีวิตจริง

- **regression test**: เปลี่ยนอะไรก็ตาม (โมเดล/chunk/threshold) → รัน evaluate() → คะแนนตก = รู้ทันที
- **เลือกโมเดลใหม่**: วัดบน golden set เรา ไม่ใช่ leaderboard (deep-technical Ch39: overfitting)
- **ข้อสอบโตได้**: query ที่เคยค้นไม่เจอ → เพิ่มเข้าชุด → แกร่งขึ้นเรื่อยๆ

Notebook: `ch11_golden_set_eval.ipynb` (execute  3 asserts) · ลีกรว่า: deep-technical Ch6/20/39/72

ewpage

บทที่ 12 (บทส่งท้าย) — Privacy & Local-First

notebook `ch12_privacy_local_first.ipynb` — พิสูจน์ว่าทั้งเล่มข้อมูลไม่เคยออกจากเครื่อง

12.1 สิ่งที่เราทำมาตลอดโดยไม่ได้พูดถึง

ทุก endpoint ที่ใช้ทั้ง 12 บท:

📁 embedding (bge-m3) http://localhost:11434 (Ollama ในเครื่อง)
📁 LLM (gemma3) http://localhost:11434
📁 vector DB ./chroma_db (ไฟล์ในเครื่อง – ไม่มี network เลย)

โน้ตส่วนตัว งานวิจัยยังไม่ตีพิมพ์ ข้อมูลนักศึกษา — ไม่มีสัปดาห์ที่ออกอินเทอร์เน็ต

12.2 ownership ที่จับต้องได้

สมองที่สอง = โพลเดอร์เดียว: ย้ายเครื่อง copy · ย้าย backup zip · ย้ายลบ ลบทิ้ง ไม่มี vendor ไม่มี subscription ไม่มี API key หมดอายุ

12.3 ต้นทุน (คำนวณใน notebook)

ที่ scale ส่วนตัว cloud ก็ไม่แพง (\$ หลักหน่วย/ปี) — เงินไม่ใช่ประเด็น privacy กับ ownership ต่างหาก · ARRA เลือก local-first ด้วยเหตุผลเดียวกัน (D1/R2 เป็นชั้น sync เสริม)

12.4 จบเล่ม — สิ่งที่ทำแล้วจริง (พิสูจน์ด้วย self-check ทั้ง 12 บท)

สร้าง second brain 20 บรรทัด → เลือก embedder เป็น (วัดด้วย GAP) → filter → เข้าใจ cosine ถึงระดับคำนวณมือ → อ่านแผนที่ความหมายเป็น → รู้จัก O(N)/ANN → สร้าง hybrid เอง → ingest idempotent → RAG+cite+abstain → ย้าย DB โดยผลไม่เปลี่ยน → วัดผลเอง (0.93 vs 0.36) → ทั้งหมด local 100%

Notebook: ch12_privacy_local_first.ipynb (execute ✅) · ลีกว่า: deep-technical Ch14/27/62/70

ewpage

บทที่ 13 — ก้าวสู่ Production: LanceDB

เปิดภาค 5 · notebook ch13_lancedb_second_brain.ipynb (execute ✅)

ChromaDB สอนเราครบทุกแนวคิด (บท 1–12) · ตอนนี้ port ไป **LanceDB** — DB ที่ ARRA ใช้จริง แนวคิดเดิม ติดตัวมาหมด เปลี่ยนแค่ API + ได้ของแถมที่ Chroma ไม่มี

13.1 เปิด second brain — embedded เหมือนเดิม

```
db = lancedb.connect('./lance_brain') # โพลเดอร์ในเครื่อง ไม่มี server
tbl = db.create_table('second_brain', data=[{'id':..., 'text':...,
'vector':...}, ...])
```

13.2 ค้นหา — ระบุ cosine ซัด (ตรงกับ ARRA lancedb.ts)

```
tbl.search(qv).distance_type('cosine').limit(2).to_pandas()
```

บทที่ 9 เจอกับดัก “Chroma default = L2” · LanceDB ก็ต้องระบุ distance_type('cosine') เอง — อย่าเชื่อ default เรื่อง metric (กฎเดิม)

13.3 filter — SQL where แทน dict

```
tbl.search(qv).where("folder = 'teaching']").limit(5)
```

บทที่ 3 ใช้ where={...} dict ของ Chroma · LanceDB ใช้ **SQL string** — power เต็ม (>, IN, AND, LIKE)

13.4 ผลรัน (self-check)

ค้น “นัดหมายกับใครบ้าง” → เจอโน้ตประชุม · filter teaching ไม่รู้ — แนวคิดบท 1-3 ทำงานบน LanceDB ทันที

13.5 ต่างจาก Chroma ตรงไหน

	Chroma (สอน)	LanceDB (production)
keyword/FTS	เขียน BM25 เอง (บท 7)	native → บท 14
versioning	×	time-travel → บท 15
runtime	Python-first	Rust core (ฝั่ง Bun/TS = ARRA)

Notebook: `ch13_lancedb_second_brain.ipynb` · ลีangkว่า: deep-technical Ch45/64

ewpage

บทที่ 14 — Hybrid ในตัว: LanceDB ทำ FTS + Vector เครื่องเดียว

notebook `ch14_lancedb_hybrid_native.ipynb` (execute )

บทที่ 7 เราเขียน BM25 เอง 20 บรรทัด + RRF เอง เพราะ Chroma ไม่มี FTS LanceDB มี **native** (engine tantivy เขียนด้วย Rust)

14.1 สร้าง FTS index — คำสั่งเดียว

```
from lancedb.index import FTS
tbl.create_index('text', config=FTS())          # แทน BM25 20 บรรทัดในบทที่ 7
tbl.search('2740', query_type='fts')            # จัดรหัสเป๊ะทันที
```

14.2 ★ Hybrid — vector + FTS + RRF ในคำสั่งเดียว

```
from lancedb.rerankers import RRFreranker
tbl.search(query_type='hybrid').vector(qv).text(q).rerank(RRFreranker()).limit(3)
```

RRF ที่เราสร้างเองบทที่ 7 (k=60) → LanceDB รวมให้ในตัว

14.3 ผลรัน (self-check)

- FTS ค้น “2740” → เจอ PR #2740 เป๊ะ
- hybrid “PR #2740” → เจอรหัส · hybrid “บอร์ดสำหรับสอน IoT” → เจอ ESP32/ไมโครคอนโทรลเลอร์

14.4 บทเรียน

เราเขียน BM25+RRF เองในบทที่ 7 เพื่อเข้าใจกลไก · production ใช้ engine ในตัว (เร็ว + scale ระดับล้าน doc) — เข้าใจก่อน แล้วค่อยใช้ของสำเร็จ = รู้ว่ามันทำอะไรอยู่ข้างใน

Notebook: `ch14_lancedb_hybrid_native.ipynb` · ลีกรกว่า: *deep-technical Ch4/11/34/56*
ewpage

บทที่ 15 (บทส่งท้าย Production) — Time-Travel: พี่เจอร์ที่ Chroma ไม่มี

notebook `ch15_lancedb_time_travel.ipynb` (execute )

LanceDB เก็บ **ทุก version** ของข้อมูล (Lance columnar) → ย้อนเวลาได้ สำคัญกับ second brain:
undo การลบผิด · audit “ตอนนั้นรู้อะไร” · reproduce งานวิจัยเก่า

15.1 ทุกการแก้ = version ใหม่

```
tbl.add([...])          # v2
tbl.delete("id='n1'")    # v3 (สมมติลบผิด!)
tbl.list_versions()       # เห็น history ครบ
```

15.2 🌟 ย้อนดู + กู้คืน (undo จริง)

```
tbl.checkout(v['version']) # ดู version เก่า
tbl.restore()              # ทำ version นั้นเป็น latest = กู้ข้อมูลที่ลบผิด
```

15.3 ผลรัน (self-check)

ลบ n1 ผิด → latest เหลือ n2, n3 → `checkout(v2) + restore()` → **n1 กลับมา** 🎉

15.4 ทำไม production เลือกสิ่งนี้

Chroma ลบแล้วหายเลย · LanceDB ย้อนได้ = ปลอดภัยกว่าสำหรับความรู้ที่ทำซ้ำไม่ได้ (deep-technical Ch45 versioning, Ch64 MVCC/WAL)

🎓 สรุปภาค Production

เรียนบน Chroma (ง่าย เห็นทุกกลไก บท 1–12) → ย้าย LanceDB ได้ทันที เพราะแนวคิดเดียวกัน + ได้ native hybrid (บท 14) + time-travel (บท 15) ของแลมระดับ production

Notebook: `ch15_lancedb_time_travel.ipynb` · ลีกรกว่า: *deep-technical Ch45/64*

ewpage

บทพิเศษ (Extras) — Vector Search ที่ Edge: Cloudflare Workers AI + Vectorize

ภาคพิเศษ · notebook `ch16_cloudflare_edge.ipynb` · grounded ใน `arra-oracle-v3/src/vector/adapters/cloudflare-vectorize.ts`

บท 1–15 รันในเครื่องเรา · บทนี้เอาแนวคิดเดิม ขึ้น **edge** ของ **Cloudflare** (300+ เมือง) โมเดลตัวเดิม (bge-m3, 1024 มิติ) แยกย้ายที่รัน

X.1 Embedding ที่ edge — Workers AI

endpoint จริง (cloudflare-vectorize.ts:53):

```
POST https://api.cloudflare.com/client/v4/accounts/{account}/ai/run/@cf/baai/bge-m3
```

bge-m3 ตัวเดียวกับ local — รันบน GPU ของ Cloudflare แทน Ollama

X.2 Vectorize — เก็บ+ค้นเวกเตอร์ที่ edge (v2 REST)

```
POST .../vectorize/v2/indexes/{index}/upsert # NDJSON, batch ≤1000
```

```
POST .../vectorize/v2/indexes/{index}/query # {vector, topK, returnMetadata}
```

⚠ index สร้างผ่าน wrangler/dashboard ไม่ใช่ runtime (comment จริง:149)

X.3 หัวใจ — ความรู้ portable

โมเดล + สมการ (cosine) + hybrid + threshold + RAG — เหมือนกันทุกอย่างระหว่าง local กับ edge
edge เปลี่ยนแค่ **ที่รัน** (เครื่องเรา → 300+ เมือง) กับ **transport** (REST แทน local call)

	Local (บท 1–15)	Edge (บทนี้)
embed	Ollama bge-m3	Workers AI @cf/baai/bge-m3
store	Chroma/LanceDB	Vectorize v2
เมื่อไหร่	personal/workshop	global/หลาย user

เหมือนบท 10 (Chroma↔LanceDB): เปลี่ยน backend ได้ ความรู้ไม่เปลี่ยน

notebook รันจริงได้ถ้าตั้ง `CF_ACCOUNT_ID + CF_API_TOKEN · grounded: deep-technical Ch5/14/69`

ewpage